

# Security Audit Report: LP Handler

---

## Solana Program Security Assessment

---

|                    |                                              |
|--------------------|----------------------------------------------|
|                    |                                              |
| Program            | DnhxA7ufxEUfR3BM2UuXWf7DK6v25JQ7CxQ6r6sJswNs |
| Prepared By        | DeGate Security Team                         |
| Date of Engagement | February 06, 2026                            |
| Version            | 3.0 (Comprehensive)                          |
| Framework          | Anchor (Rust)                                |
| Blockchain         | Solana                                       |

---

## Table of Contents

---

- [1. Executive Overview](#)
  - [2. Risk Methodology \(BVSS\)](#)
  - [3. Security Architecture Assessment](#)
  - [4. Asset Flow Constraint Validation](#)
  - [5. Verified Findings](#)
  - [6. Detailed Findings](#)
  - [7. SlowMist Best Practices Compliance](#)
  - [8. Handler-Level Security Assessment](#)
  - [9. Positive Security Observations](#)
  - [10. Operational Security Recommendations](#)
  - [11. Recommendations Summary](#)
  - [12. Error Code Reference](#)
  - [Appendix A: Files Analyzed](#)
  - [Appendix B: Security Invariants Summary](#)
  - [Appendix C: Hardcoded Whitelists](#)
  - [Appendix D: State Structures](#)
  - [Appendix E: Automated Scanner False Positive Summary](#)
  - [Appendix F: Agent Analysis Summary](#)
  - [Appendix G: Disclaimer](#)
-

## 1. Executive Overview

---

### 1.1 Introduction

This document presents the consolidated security audit report for the **DeGate LP Handler** Solana program. This report synthesizes findings from:

1. **Deep Automated Analysis** -- SolanaGuard (44 patterns) AST + Regex scanning
2. **Multi-Agent Deep Analysis** -- Specialized agents for CPI, Account Validation, Arithmetic, and Security Layer analysis
3. **Handler-Level Assessment** -- Per-instruction account constraints, state transitions, and permission matrix review
4. **Checklist** -- Common pitfalls of Solana smart contracts
5. **Audit Requirements Validation** -- Direct validation against the [DeGate LP Handler Audit Requirements](#) scope

The LP Handler implements a novel "**Entry Snapshot + Exit Reconciliation**" security model (the `secure_entrypoint!` macro) designed to ensure that no permission residuals or asset anomalies can occur during transaction execution, regardless of how the underlying business logic is implemented.

### 1.2 Assessment Summary

#### Overall Risk Assessment

| Metric                   | Value                             |
|--------------------------|-----------------------------------|
| Risk Level               | HIGH                              |
| Security Score           | 62/100                            |
| Total Verified Findings  | 21                                |
| Critical                 | 2                                 |
| High                     | 6                                 |
| Medium                   | 7                                 |
| Low                      | 6                                 |
| Files In Scope           | 12                                |
| Lines Analyzed           | ~3,576                            |
| Checklist Items          | 27 (22 Safe, 2 Issues, 3 Notes)   |
| False Positives Filtered | 107 (from 117 automated findings) |

## Audit Scope

Per the audit requirements document, the scope is divided as follows:

### In Scope (Audited):

| File Path                    | Priority | Description                                                |
|------------------------------|----------|------------------------------------------------------------|
| <code>security/mod.rs</code> | CRITICAL | Core Entry Snapshot + Exit Reconciliation logic            |
| <code>lib.rs</code>          | HIGH     | Program entry point; <code>secure_entrypoint!</code> macro |
| <code>state/</code>          | MEDIUM   | Constants, state definitions, error codes, utilities       |

### Out of Scope (Not Audited for Business Logic Correctness):

| File Path                  | Reason                                                                            |
|----------------------------|-----------------------------------------------------------------------------------|
| <code>instructions/</code> | Business logic; security model designed to contain vulnerabilities in this module |
| <code>tests/</code>        | Test code; not deployed                                                           |

**Note:** `instructions/` files were still reviewed for account constraint definitions, CPI patterns, and security layer integration, but not for business logic correctness.

## 1.3 Methodology

### Phase 1: Automated Scanning

- AST-based semantic analysis using tree-sitter
- 44 vulnerability pattern regex matching
- Initial findings: 117 (38 critical, 79 high)
- After false positive filtering: 10 true positives

### Phase 2: Multi-Agent Deep Analysis

Four specialized agents analyzed different security domains in parallel:

- **CPI Security Agent** -- Cross-program invocation patterns and remaining accounts handling
- **Account Validation Agent** -- Account constraints, ownership checks, `UncheckedAccount` usage
- **Arithmetic Agent** -- Mathematical operations, precision loss, overflow/underflow
- **Security Layer Agent** -- Entry/exit check implementation gaps, snapshot completeness

### Phase 3: Best Practices Compliance

Full 27-item checklist:

- 15 Common Pitfalls of Solana Smart Contracts
- 12 Attacks Using the Anchor Framework

### Phase 4: Audit Requirements Validation

Direct verification of each focus area from the audit requirements:

1. Snapshot Completeness
2. Reconciliation Correctness
3. Approval/Delegation Controls
4. Whitelist Integrity
5. Boundary Enforcement
6. Token-2022 Specific Concerns

### Phase 5: Manual Expert Verification

- Code-level review of each finding
- Exploit scenario development
- Cross-referencing between findings from different phases

---

## 2. Risk Methodology (BVSS)

### 2.1 Severity Classification

| Score Range | Severity |
|-------------|----------|
| 9.0 -- 10.0 | CRITICAL |
| 7.0 -- 8.9  | HIGH     |
| 4.0 -- 6.9  | MEDIUM   |
| 0.1 -- 3.9  | LOW      |

### 2.2 BVSS Calculation

**BVSS Score = Exploitability x Impact**

- **Exploitability** = Attack Origin x Attack Cost x Attack Complexity
- **Impact** = max(Impact on Deposits, Impact on Withdrawals, Impact on Governance)

### 3. Security Architecture Assessment

This section validates the core security model against the audit requirements (Section 3 and 5.3).

#### 3.1 Core Security Model: Entry Snapshot + Exit Reconciliation

The `secure_entrypoint!` macro (defined at `lib.rs:27-54`) enforces a four-phase execution pattern:

1. **Phase 1:** `resolve_policy()` -- Load SecurityPolicy (hardcoded defaults)
2. **Phase 2:** `entry_check_and_snapshot()` -- Pre-execution validation + state snapshot
3. **Phase 3:** `<business logic body>` -- Instruction handler execution
4. **Phase 4:** `exit_check()` -- Post-execution reconciliation

SecurityPolicy ( `security/mod.rs:9-22` )

| Policy Flag                         | Default           | Effect                                                       |
|-------------------------------------|-------------------|--------------------------------------------------------------|
| <code>forbid_delegate</code>        | <code>true</code> | Token accounts must not have delegate after execution        |
| <code>forbid_close_authority</code> | <code>true</code> | Token accounts must not have close_authority after execution |

#### Boundary Enforcement Verification (Audit Req 5.3.5)

By macro design, the `secure_entrypoint!` macro captures `ctx.accounts.to_account_infos()` BEFORE the body executes, and `exit_check()` runs AFTER the body returns. Since the macro is expanded inline and the body is a single expression, there is no way for the body to skip or modify the exit check. All three business instructions ( `swap_and_deposit` , `decrease_liquidity` , `increase_liquidity` ) use this macro.

Admin instructions ( `init_security_config` , `update_security_config` , `close_security_config` ) intentionally skip the security sandwich. This is by design since the `security_config` PDA may not exist when these are called. These instructions only modify the `security_config` PDA and are guarded by authority checks.

### 3.2 Snapshot Completeness Assessment (Audit Req 5.3.1)

#### What the Entry Snapshot Captures

| Data Captured                                                | Source                                | Scope                                                                     |
|--------------------------------------------------------------|---------------------------------------|---------------------------------------------------------------------------|
| Signer token accounts (delegate, close_authority, wSOL flag) | <code>security/mod.rs: 313-344</code> | <code>ctx.accounts</code> where <code>authority == signer</code>          |
| Signer token accounts from remaining_accounts                | <code>security/mod.rs: 387-403</code> | <code>remaining_accounts</code> where <code>authority == signer</code>    |
| Uninitialized account indices                                | <code>security/mod.rs: 331-333</code> | <code>ctx.accounts</code> where <code>system_owner + data_len == 0</code> |

#### Gap Analysis

| Aspect                               | Covered? | Notes                                                                                                                         |
|--------------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------|
| Signer token account authorities     | Yes      | Both <code>ctx.accounts</code> and <code>remaining_accounts</code>                                                            |
| Signer token account delegate        | Yes      | Captured for exit comparison                                                                                                  |
| Signer token account close_authority | Yes      | Captured for exit comparison                                                                                                  |
| Recipient token accounts             | Partial  | Only validated as "allowed authority" in token whitelist check; not snapshotted independently                                 |
| Uninitialized accounts               | Yes      | Indices stored for exit new-account validation                                                                                |
| Non-signer token account balances    | No       | Not tracked; relies on whitelist-only constraint                                                                              |
| LP NFT ownership                     | Indirect | Enforced via Anchor constraints in <code>increase_liquidity</code> , but NOT in <code>decrease_liquidity</code> (see LPH-006) |

**Finding:** The snapshot does NOT independently track `recipient` token account state. This is acceptable given the trust model (recipient is assumed authorized by frontend), but see LPH-002 and LPH-007 for risks.

### 3.3 Reconciliation Correctness Assessment (Audit Req 5.3.2)

Exit Checks Implemented ( `security/mod.rs:417-527` )

| Check                        | Line    | Description                                                            | Verdict                             |
|------------------------------|---------|------------------------------------------------------------------------|-------------------------------------|
| SOL leakage prevention       | 440-448 | Program-owned accounts <code>lamports &lt;= rent-exempt minimum</code> | CORRECT                             |
| Token authority unchanged    | 461-466 | <code>cur.owner == signer</code>                                       | CORRECT                             |
| Delegate forbidden           | 467-480 | Default: <code>cur.delegate.is_none()</code>                           | CORRECT (see LPH-003 for entry gap) |
| Close authority forbidden    | 482-496 | Default: <code>cur.close_authority.is_none()</code>                    | CORRECT (see LPH-003 for entry gap) |
| New account owner validation | 508-514 | Owner in <code>ALLOWED_ACCOUNT_OWNERS</code>                           | CORRECT                             |
| New token account authority  | 515-522 | Authority must be signer or recipient                                  | CORRECT                             |
| wSOL ATA closure exception   | 456-458 | wSOL ATA allowed to become uninitialized                               | CORRECT (necessary for unwrap flow) |

#### Gap Analysis

| Potential Bypass                        | Mitigated? | Notes                                               |
|-----------------------------------------|------------|-----------------------------------------------------|
| Asset leaves to non-whitelisted account | Yes        | Token account authority whitelist at entry          |
| New approval created post-execution     | Yes        | Exit check enforces <code>delegate.is_none()</code> |
| SOL drained from program accounts       | Yes        | Lamport leakage check                               |
| Account substitution between entry/exit | Partial    | See LPH-005: account key not re-verified            |

### 3.4 Approval/Delegation Controls (Audit Req 5.3.3)

#### LP NFT (Token-2022) approval checking:

- Exit check enforces `delegate.is_none()` and `close_authority.is_none()` for all signer token accounts (including NFT)
- **Gap:** Pre-existing delegates at entry are NOT rejected (see LPH-003)
- **Gap:** LP NFT in `decrease_liquidity` is `UncheckedAccount` (see LPH-006)

#### Can approvals persist after transaction?

- No, the default policy `forbid_delegate = true` ensures delegate is None after exit
- However, if the policy were changed to `forbid_delegate = false`, pre-existing delegates would persist

### 3.5 Whitelist Integrity (Audit Req 5.3.4)

#### Pool Whitelist:

- Stored in `security_config` PDA (seeds: `[b"security_config"]`)
- Maximum 40 pools (`MAX_ALLOWED_POOLS`)
- Must be non-empty; pool\_state key validated against whitelist
- PDA owner validated as `crate::ID`
- Cannot be manipulated by callers (only admin can update)
- **Verdict:** SECURE

#### Executable Program Whitelist:

- Hardcoded in `state/consts.rs:12-20` (7 programs)
- Cannot be modified at runtime
- **Verdict:** SECURE

#### Fee Owner Whitelist:

- Stored in `security_config` PDA alongside pools
- Maximum 6 fee owners (`MAX_FEE_OWNERS`)
- **Gap:** The `fee_owner` SystemAccount parameter is NOT validated against this whitelist (see LPH-001)

#### Account Owner Whitelist:

- Hardcoded in `state/consts.rs:25-31` (5 programs)
- Applied to newly initialized accounts at exit
- **Verdict:** SECURE

### 3.6 Token-2022 Specific Assessment (Audit Req 5.3.6)

#### Extension features NOT checked by the security model:

| Extension            | Risk                                                        | Mitigation                                 |
|----------------------|-------------------------------------------------------------|--------------------------------------------|
| TransferHook         | Hook program could execute arbitrary logic during transfers | Pool whitelist (only admin-approved pools) |
| PermanentDelegate    | Delegate can drain tokens even after exit check             | Pool whitelist                             |
| ConfidentialTransfer | Could hide real balances                                    | Pool whitelist                             |
| NonTransferable      | Could block expected transfers                              | Pool whitelist                             |

**Note:** The error code `SecurityToken2022ForbiddenExtension` ( `errors.rs:79-80` ) is defined but **no implementation** checks for these extensions. The pool whitelist mechanism is the sole defense.

**Account approval/delegation checks:** Implemented and enforced via exit checks. The `parse_token2022_account` function ( `security/mod.rs:115-133` ) correctly extracts delegate and `close_authority` from Token-2022 extension state.

**Verdict:** Token-2022 extension risk is deferred to the pool whitelist mechanism. This is acceptable given the documented assumption that pools are admin-curated, but introduces operational risk if a whitelisted pool's token adds dangerous extensions post-whitelist.

## 4. Asset Flow Constraint Validation

Per audit requirements Section 4, assets must flow ONLY between four whitelisted account categories.

### 4.1 Whitelisted Account Categories

| Category | Account Type                       | Validation Method                                            | Verdict                 |
|----------|------------------------------------|--------------------------------------------------------------|-------------------------|
| 1.1      | <code>signer</code>                | Anchor <code>Signer&lt;'info&gt;</code> type; entry snapshot | SECURE                  |
| 1.2      | <code>recipient</code>             | UncheckedAccount; NO on-chain validation                     | <b>GAP</b><br>(LPH-002) |
| 1.3      | <code>fee_owner</code>             | SystemAccount; NO whitelist validation                       | <b>GAP</b><br>(LPH-001) |
| 1.4      | Third-party DAPP / System Accounts | Executable whitelist + account owner whitelist               | SECURE                  |

## 4.2 Protected Asset Categories

| Asset                        | Flow Constraint                         | Enforcement                                    | Verdict                                |
|------------------------------|-----------------------------------------|------------------------------------------------|----------------------------------------|
| LP Pool Token1               | Only whitelisted accounts               | Token account authority validation at entry    | SECURE                                 |
| LP Pool Token2               | Only whitelisted accounts               | Token account authority validation at entry    | SECURE                                 |
| LP Position NFT (Token-2022) | Only signer/recipient; no new approvals | Exit check: authority unchanged, delegate=None | SECURE (with LPH-003, LPH-006 caveats) |
| Reward Tokens                | Only whitelisted accounts               | Token account authority validation at entry    | SECURE                                 |

## 4.3 Security Invariant Assessment

**Stated Invariant:** Under the assumption of non-malicious third-party DAPPs, all assets ultimately reside ONLY in accounts controlled by the user (signer/recipient) or DeGate (fee\_owner).

**Assessment:** The invariant holds for signer-controlled accounts due to comprehensive exit checks. However, the invariant has gaps for:

1. **Recipient** (Category 1.2) -- No on-chain validation means assets could flow to an uncontrolled address
2. **Fee Owner** (Category 1.3) -- No whitelist validation means fees could flow to an attacker's address

These gaps do not break the invariant IF the documented prerequisite holds (frontend validates recipient, fee\_owner is always legitimate). But they represent on-chain enforcement gaps.

---

## 5. Verified Findings

### 5.1 Findings Summary

**IMPORTANT:** All findings below are marked **REQUIRES REVIEW**. This automated analysis may have missed mitigations present in other code paths. Each finding must be manually verified by the development team. Additionally, each finding should have corresponding **test cases** written to cover the described edge cases and exploit scenarios, ensuring ongoing regression protection.

| ID      | Title                                               | Severity | BVSS | Status          | Category           |
|---------|-----------------------------------------------------|----------|------|-----------------|--------------------|
| LPH-001 | Unvalidated fee_owner Address                       | CRITICAL | 10.0 | REQUIRES REVIEW | Account Validation |
| LPH-002 | Unvalidated recipient Address                       | CRITICAL | 8.5  | REQUIRES REVIEW | Account Validation |
| LPH-003 | Pre-existing Token Delegates Not Rejected at Entry  | HIGH     | 7.8  | REQUIRES REVIEW | Security Layer     |
| LPH-004 | Arithmetic Masking via saturating_sub               | HIGH     | 7.2  | REQUIRES REVIEW | Arithmetic         |
| LPH-005 | Account Identity Not Re-verified in Exit Check      | HIGH     | 7.0  | REQUIRES REVIEW | Security Layer     |
| LPH-006 | Position NFT Account Inconsistent Validation        | HIGH     | 7.0  | REQUIRES REVIEW | Account Validation |
| LPH-007 | Recipient Parameter Not Verified in Security Layer  | HIGH     | 6.8  | REQUIRES REVIEW | Security Layer     |
| LPH-008 | Division Truncation in Reward Split                 | MEDIUM   | 5.5  | REQUIRES REVIEW | Arithmetic         |
| LPH-009 | Fixed 100-Unit Buffer in Min Output                 | MEDIUM   | 4.5  | REQUIRES REVIEW | Arithmetic         |
| LPH-010 | Fee Calculation Precision Loss                      | MEDIUM   | 4.0  | REQUIRES REVIEW | Arithmetic         |
| LPH-011 | init_security_config Has No Admin Gate              | MEDIUM   | 5.0  | REQUIRES REVIEW | Governance         |
| LPH-012 | Authority Key Not Rotatable                         | MEDIUM   | 4.5  | REQUIRES REVIEW | Governance         |
| LPH-013 | Token-2022 Extension Checks Not Implemented         | MEDIUM   | 4.0  | REQUIRES REVIEW | Token-2022         |
| LPH-014 | close_security_config Blocks All Operations         | MEDIUM   | 5.0  | REQUIRES REVIEW | Governance         |
| LPH-015 | No Events on Security Config Changes                | LOW      | 2.5  | REQUIRES REVIEW | Observability      |
| LPH-016 | <code>.unwrap()</code> Instead of Error Propagation | LOW      | 2.0  | REQUIRES REVIEW | Code Quality       |

| ID      | Title                                                        | Severity | BVSS | Status          | Category       |
|---------|--------------------------------------------------------------|----------|------|-----------------|----------------|
| LPH-017 | Direct Multiplication Without <code>checked_mul</code>       | LOW      | 1.5  | REQUIRES REVIEW | Arithmetic     |
| LPH-018 | <code>unwrap_or(0)</code> Pattern Masks Errors               | LOW      | 1.0  | REQUIRES REVIEW | Code Quality   |
| LPH-019 | Caller-Controlled <code>fee_percent</code> Allows Fee Bypass | HIGH     | 7.5  | REQUIRES REVIEW | Business Logic |
| LPH-020 | Fee Rounding Exploitable via Micro-Transactions              | LOW      | 2.0  | REQUIRES REVIEW | Arithmetic     |
| LPH-021 | Missing overflow-checks in Release Profile                   | LOW      | 1.5  | REQUIRES REVIEW | Build Config   |

## 6. Detailed Findings

### 6.1 LPH-001: Unvalidated `fee_owner` Address

|          |                    |
|----------|--------------------|
|          |                    |
| Severity | CRITICAL           |
| BVSS     | 10.0               |
| Status   | REQUIRES REVIEW    |
| Category | Account Validation |

#### Description

The `fee_owner` account is accepted as an unconstrained `SystemAccount` without validating it belongs to the `fee_owners` whitelist stored in `security_config`. The code comment states runtime validation in the handler, but **no such runtime validation exists**.

The security layer at `security/mod.rs:346-357` validates that **token accounts** have their authority in the `fee_owners` list, but does **NOT** validate that the `fee_owner` `SystemAccount` parameter itself is whitelisted.

#### Code Location

File: `instructions/decrease_liquidity.rs:76-80`

```

/// ... handler-level runtime validation (see require! below).
#[account(mut)]
pub fee_owner: SystemAccount<'info>,

```

**File:** `security/mod.rs:346-357` (gap)

```

} else if ta.owner != recipient {
    let is_fee_owner = if fee_owners_n != 0 {
        let cfg_ai = cfg_ai_for_fee.as_ref().unwrap();
        let data = cfg_ai.data.borrow();
        security_config_fee_owner_contains(&data, pools_n, fee_owners_n, &ta.o
wner)
    }
    //
    ^^^^^^^^^^
    // Validates TOKEN ACCOUNT authority, NOT the fee_owner parameter
    itself!
    } else {
        false
    };
};

```

## Exploit Scenario

1. Attacker crafts a transaction with their own address as `fee_owner`
2. Creates corresponding `fee_token0_account` and `fee_token1_account` owned by their address
3. Security layer passes: it only validates token account ownership relationships, not the `fee_owner` `SystemAccount`
4. All protocol fees (SOL system transfers and token transfers) go to attacker
5. Attacker drains all protocol fees indefinitely

## BVSS Score

| Metric            | Value           |
|-------------------|-----------------|
| Attack Origin     | Arbitrary (1.0) |
| Attack Cost       | Low (1.0)       |
| Attack Complexity | Low (1.0)       |
| Impact            | Critical (1.0)  |
| <b>BVSS Score</b> | <b>10.0</b>     |

## Recommendation

Add explicit validation in the handler before any fee transfer:

```
// In decrease_liquidity.rs, after parameter validation:
let cfg_data = ctx.accounts.security_config.try_borrow_data()?;
let pools_n = security_config_pools_len(&cfg_data)?;
let fee_n = security_config_fee_owners_len(&cfg_data, pools_n)?;
require!(
    security_config_fee_owner_contains(&cfg_data, pools_n, fee_n, &ctx.accounts.fee_owner.key()),
    LpDepositError::InvalidFeeOwner
);
```

## Review Notes

This finding is based on static analysis. The development team should verify whether there is any additional fee\_owner validation in the handler body or in the CPI call path that this analysis may have missed. Check all code paths that invoke `transfer_fee()` or `system_program::transfer` to the fee\_owner.

## Required Tests

1. **Non-whitelisted fee\_owner is rejected** -- Call `decrease_liquidity` with a `fee_owner` address NOT in `security_config.fee_owners`. Expect the transaction to fail.
2. **Attacker-owned fee\_token accounts are rejected** -- Create `fee_token0_account` and `fee_token1_account` with an attacker's authority. Verify fees cannot be redirected.
3. **Valid fee\_owner in whitelist succeeds** -- Call `decrease_liquidity` with a fee\_owner that IS in the whitelist. Verify fees go to the correct address.
4. **fee\_owner == signer is rejected** -- Attempt to set `fee_owner` to the same address as `signer` to bypass fee collection.
5. **fee\_owner == recipient is rejected** -- Attempt to use `recipient` address as `fee_owner`.

## 6.2 LPH-002: Unvalidated recipient Address

|          |                    |
|----------|--------------------|
|          |                    |
| Severity | CRITICAL           |
| BVSS     | 8.5                |
| Status   | REQUIRES REVIEW    |
| Category | Account Validation |

## Description

The `recipient` account is an `UncheckedAccount` with only a comment stating frontend must validate. There is **NO on-chain validation** that the recipient is legitimate, a signer, or authorized by the transaction signer. The recipient receives the LP Position NFT (in `swap_and_deposit` ) and withdrawn funds (in `decrease_liquidity` ).

## Code Location

**File:** `instructions/decrease_liquidity.rs:36-38`

```
/// CHECK: recipient for withdraw and claim ... frontend must validate
recipient address
#[account(mut)]
pub recipient: UncheckedAccount<'info>,
```

**File:** `security/mod.rs:269-274` (no verification)

```
pub fn entry_check_and_snapshot<'info>(
    ...
    recipient: Pubkey,    // Accepted without verification
    ...
```

## Exploit Scenario

1. User initiates a legitimate withdrawal
2. Transaction is intercepted (MEV bot, malicious RPC, frontend compromise)
3. Attacker modifies `recipient` to their own wallet
4. All withdrawn tokens/SOL transferred to attacker
5. Security layer passes because `recipient` is treated as a trusted parameter

## BVSS Score

| Metric            | Value           |
|-------------------|-----------------|
| Attack Origin     | Arbitrary (1.0) |
| Attack Cost       | Medium (0.67)   |
| Attack Complexity | Medium (0.67)   |
| Impact            | Critical (1.0)  |
| <b>BVSS Score</b> | <b>8.5</b>      |

## Recommendation

**Option 1 (Strongest):** Require recipient to be a co-signer:

```
pub recipient: Signer<'info>,
```

**Option 2:** Validate `recipient == signer` by default, with opt-in separation:

```
require!(
  ctx.accounts.recipient.key() == ctx.accounts.signer.key()
  || is_authorized_recipient(&ctx.accounts.signer.key(), &ctx.accounts.recipient.key()),
  LpDepositError::InvalidRecipient
);
```

## Review Notes

This finding assumes the frontend/wallet is the only recipient validation. The development team should verify whether the recipient address is validated in any middleware, SDK, or wallet integration layer not visible in the on-chain code. Also check if the `increase_liquidity` instruction (where `recipient = signer` is enforced at `lib.rs:141`) pattern should be applied to other instructions.

## Required Tests

1. **Arbitrary recipient receives funds** -- Call `decrease_liquidity` with `recipient` set to a random address. Verify whether withdrawn tokens/SOL arrive at that address.
2. **recipient != signer in swap\_and\_deposit** -- Call `swap_and_deposit` with a `recipient` different from `signer`. Verify where the LP Position NFT is minted to.
3. **recipient is zero address** -- Pass the zero address ( `11111111111111111111111111111111` ) as recipient. Verify the transaction fails or handles it gracefully.
4. **recipient modification in signed transaction** -- Simulate a scenario where recipient is changed after signer approves. Verify whether the security layer catches it.
5. **recipient == signer succeeds normally** -- Confirm the normal flow where recipient equals signer works correctly.

### 6.3 LPH-003: Pre-existing Token Delegates Not Rejected at Entry

|          |                 |
|----------|-----------------|
|          |                 |
| Severity | HIGH            |
| BVSS     | 7.8             |
| Status   | REQUIRES REVIEW |
| Category | Security Layer  |

#### Description

When `policy.forbid_delegate == true` (the default), the exit check ensures `delegate.is_none()`. However, the **entry check does NOT reject** accounts that already have a delegate set. The entry phase only snapshots the existing delegate for later comparison.

#### Code Location

**File:** `security/mod.rs:334-344` (entry -- missing validation)

```
if let Some(ta) = parse_token_account(ai) {
    if ta.owner == signer {
        signer_token_accounts.push(SignerTokenAccountBefore {
            delegate: ta.delegate,           // Captured but NOT validated
            close_authority: ta.close_authority, // Captured but NOT validated
            ...
        });
    }
}
```

**File:** `security/mod.rs:467-480` (exit -- validates None)

```
if policy.forbid_delegate {
    require_log!(cur.delegate.is_none(), ...); // Only checks post-execution
    state
}
```

#### Analysis

Under the default policy ( `forbid_delegate = true` ), the exit check correctly requires `delegate.is_none()`. This means the business logic must clear any pre-existing delegate during execution. If a pre-existing delegate exists and the business logic does not clear it, the exit check will fail -- which is safe.

However, if the policy were ever changed to `forbid_delegate = false`, the exit check only verifies `cur.delegate == b.delegate`, which would allow a pre-existing malicious delegate to persist.

## Recommendation

Add entry-phase validation:

```
if policy.forbid_delegate && ta.delegate.is_some() {
    return err!(LpDepositError::SecurityTokenHasExistingDelegate);
}
if policy.forbid_close_authority && ta.close_authority.is_some() {
    return err!(LpDepositError::SecurityTokenHasExistingCloseAuthority);
}
```

## Review Notes

Under the default policy ( `forbid_delegate = true` ), the exit check does correctly enforce `delegate.is_none()` , which means any pre-existing delegate would cause the exit check to fail -- making this finding safe under default policy. However, verify that the default policy can never be changed to `forbid_delegate = false` in production without understanding this implication. Also verify whether any business logic within the instruction body could clear a pre-existing delegate during execution.

## Required Tests

1. **Token account with pre-existing delegate at entry** -- Set a delegate on a signer's token account, then call any business instruction. Verify the exit check fails with `SecurityTokenDelegateNotAllowed` .
2. **Token account with pre-existing close\_authority at entry** -- Set a close\_authority on a signer's token account, then call any business instruction. Verify the exit check fails.
3. **Default policy forbid\_delegate = true blocks delegate** -- Confirm that the default policy correctly rejects accounts where the instruction body sets a delegate.
4. **Verify policy cannot be changed at runtime** -- Confirm that `resolve_policy()` always returns hardcoded defaults and cannot be influenced by caller input.
5. **remaining\_accounts token with delegate** -- Pass a signer-owned token account with a pre-existing delegate via `remaining_accounts` . Verify it is caught.

---

## 6.4 LPH-004: Arithmetic Masking via saturating\_sub

| Severity | HIGH            |
|----------|-----------------|
| BVSS     | 7.2             |
| Status   | REQUIRES REVIEW |
| Category | Arithmetic      |

## Description

The reward calculation uses `saturating_sub` instead of `checked_sub`, silently returning 0 on underflow. This masks cases where `principal_expected > actual_delta`, leading to zero reward and zero fee.

## Code Location

**File:** `instructions/decrease_liquidity.rs:288-289`

```
let reward_gross_0 =  
signer_token0_amount.saturating_sub(principal_expected_0);  
let reward_gross_1 = signer_token1_amount.saturating_sub(principal_expected_1)  
;
```

## Impact

| Scenario                           | Expected                   | Actual                                                |
|------------------------------------|----------------------------|-------------------------------------------------------|
| <code>actual &lt; principal</code> | Error or explicit handling | reward = 0 silently                                   |
| Fee on 0 reward                    | Should be flagged          | fee = 0 silently                                      |
| Timing attack                      | N/A                        | User times withdrawal during volatility to avoid fees |

The README acknowledges this as intentional (`saturating_sub` prevents negative reward and avoids charging principal). However, it creates an exploitable pattern where users can time withdrawals during price volatility to minimize or eliminate fees.

## Recommendation

Replace with explicit handling:

```
let reward_gross_0 = if signer_token0_amount >= principal_expected_0 {  
    signer_token0_amount - principal_expected_0  
} else {  
    msg!("WARN: actual < principal for token0, reward=0");  
    0  
};
```

## Review Notes

The README documents this as intentional behavior -- `saturating_sub` prevents charging fees on principal when `actual < expected` due to price movement. The development team should evaluate whether this is an acceptable trade-off (no fee on actual rewards in edge cases) or an exploitable pattern. Check how frequently `principal_expected > actual_delta` can occur in practice with real Raydium CLMM pools.

## Required Tests

1. **reward\_gross = 0 when actual < principal** -- Create a scenario where pool price moves such that `signer_token_amount < principal_expected`. Verify `reward_gross = 0` and `fee = 0`.
2. **Normal reward calculation** -- Withdraw with `actual > principal`. Verify `reward_gross = actual - principal` and fee is calculated correctly.
3. **Boundary case actual == principal** -- Verify `reward = 0` and `fee = 0`.
4. **Large price movement fee avoidance** -- Simulate extreme volatility where a user times withdrawal to make `actual < principal` on one token. Verify total fee across both tokens is correct.
5. **fee\_percent = 0 edge case** -- Verify no division issues when `fee_percent` is 0.
6. **fee\_percent = 10000 (100%) edge case** -- Verify all reward goes to `fee_owner`.

---

## 6.5 LPH-005: Account Identity Not Re-verified in Exit Check

| Severity | HIGH            |
|----------|-----------------|
| BVSS     | 7.0             |
| Status   | REQUIRES REVIEW |
| Category | Security Layer  |

### Description

The exit check validates accounts at indices stored during entry but does not verify the account pubkey hasn't been swapped. While Solana's runtime makes account substitution within a single instruction difficult, storing and re-verifying keys provides defense-in-depth.

### Code Location

File: `security/mod.rs:500-506`

```

for idx in before.uninitialized_indices.iter().copied() {
    let ai = accounts.get(idx)
        .ok_or(LpDepositError::SecurityAccountSetChanged)?;
    if is_uninitialized_account(ai) {
        continue;
    }
    // Account key NOT verified to match entry snapshot

```

## Recommendation

Store keys alongside indices:

```

pub uninitialized_accounts: Vec<(usize, Pubkey)>, // Index + key

// In exit_check:
require_keys_eq!(ai.key(), *original_key, LpDepositError::SecurityAccountKeyCh
anged);

```

## Review Notes

Solana's runtime guarantees that account references within a single instruction invocation are stable (the account list is fixed at instruction entry). Verify whether the `accounts` slice passed to `exit_check` is the same reference as the one passed to `entry_check_and_snapshot`. If they reference the same underlying account array, substitution is impossible at the Solana runtime level. This finding is a defense-in-depth concern rather than a direct exploit.

## Required Tests

1. **Uninitialized account becomes initialized at exit** -- Pass an uninitialized account at entry, have the business logic initialize it via CPI. Verify the exit check validates its owner against `ALLOWED_ACCOUNT_OWNERS`.
2. **Newly initialized token account with wrong authority** -- Create a new token account during execution with authority != signer and != recipient. Verify exit check fails with `SecurityNewTokenAccountAuthorityInvalid`.
3. **Newly initialized account with disallowed owner** -- Have a CPI create an account owned by a non-whitelisted program. Verify exit check fails with `SecurityDisallowedAccountOwner`.
4. **Account remains uninitialized** -- Verify that accounts still uninitialized at exit are silently skipped (expected behavior).

## 6.6 LPH-006: Position NFT Account Inconsistent Validation

|          |                    |
|----------|--------------------|
|          |                    |
| Severity | HIGH               |
| BVSS     | 7.0                |
| Status   | REQUIRES REVIEW    |
| Category | Account Validation |

### Description

`position_nft_account` is `UncheckedAccount` in `decrease_liquidity.rs` but properly validated in `increase_liquidity.rs`. This inconsistency means `decrease_liquidity` accepts any account as the position NFT.

### Code Location

**File:** `instructions/decrease_liquidity.rs:159-161` (VULNERABLE)

```
/// CHECK: position NFT ...  
#[account(mut)]  
pub position_nft_account: UncheckedAccount<'info>,
```

**File:** `instructions/increase_liquidity.rs:56-61` (CORRECT)

```
#[account(  
    constraint = position_nft_account.mint == personal_position.nft_mint,  
    constraint = position_nft_account.amount == 1,  
    token::authority = signer  
)]  
pub position_nft_account: Box<InterfaceAccount<'info, TokenAccount>>,
```

### Recommendation

Apply consistent validation:

```
#[account(  
    mut,  
    constraint = position_nft_account.mint == personal_position.nft_mint,  
    constraint = position_nft_account.amount == 1,  
    token::authority = signer  
)]  
pub position_nft_account: Box<InterfaceAccount<'info, TokenAccount>>,
```

## Review Notes

Verify whether Raydium's `decrease_liquidity_v2` CPI already validates the `position_nft_account` internally (e.g., by checking the NFT mint against the position). If Raydium provides this validation, the risk is reduced to a defense-in-depth concern. However, the inconsistency between `increase_liquidity` (validated) and `decrease_liquidity` (unvalidated) should be resolved regardless.

## Required Tests

1. **Pass wrong token account as position\_nft\_account** -- Call `decrease_liquidity` with a token account that does NOT hold the position NFT. Verify the transaction fails.
2. **Pass arbitrary account as position\_nft\_account** -- Call `decrease_liquidity` with a non-token account. Verify the transaction fails.
3. **Pass position\_nft\_account with amount != 1** -- Use a token account with amount=0 or amount>1. Verify it is rejected.
4. **Pass position\_nft\_account with wrong authority** -- Use a token account holding the correct NFT but owned by a different user. Verify rejection.
5. **Valid position\_nft\_account succeeds** -- Confirm the normal flow with correct NFT token account works.

---

## 6.7 LPH-007: Recipient Parameter Not Verified in Security Layer

| Severity | HIGH            |
|----------|-----------------|
| BVSS     | 6.8             |
| Status   | REQUIRES REVIEW |
| Category | Security Layer  |

### Description

The `recipient` Pubkey passed to `entry_check_and_snapshot` is used for authorization decisions (allowing recipient-owned token accounts, allowing new token accounts with recipient authority) but is never verified to exist in the accounts array or have any specific properties.

### Code Location

File: `security/mod.rs:269-274`

```
pub fn entry_check_and_snapshot<'info>(  
    ...  
    recipient: Pubkey,    // Used in authorization but never verified
```

At line 346:

```
} else if ta.owner != recipient {  
    // Non-signer/non-recipient token accounts get special validation
```

An arbitrary `recipient` value widens the set of "allowed" token account authorities.

### Recommendation

```
let recipient_exists = accounts.iter().any(|a| a.key() == recipient);  
require!(recipient_exists, LpDepositError::RecipientNotInAccounts);
```

### Review Notes

Verify the full set of places where `recipient` is used in authorization decisions within `security/mod.rs`. An arbitrary recipient value could widen the "allowed" token account authority set, potentially letting the security layer accept token accounts it should reject. Check whether `increase_liquidity` (where `recipient = signer`) is immune to this issue.

### Required Tests

1. **Fabricated recipient widens token account acceptance** -- Pass a `recipient` address equal to the authority of a non-signer token account. Verify whether the security layer incorrectly accepts that token account.
2. **recipient that doesn't exist in accounts array** -- Pass a pubkey as `recipient` that is not present in any account in the transaction. Verify the impact on authorization decisions.
3. **increase\_liquidity always uses signer as recipient** -- Confirm that `lib.rs:141` correctly forces `recipient = signer` and that the security layer cannot be widened.
4. **recipient matches a fee\_owner address** -- Pass a recipient that equals one of the `fee_owners`. Verify correct behavior.

---

## 6.8 LPH-008: Division Truncation in Reward Split

| Severity | MEDIUM          |
|----------|-----------------|
| BVSS     | 5.5             |
| Status   | REQUIRES REVIEW |
| Category | Arithmetic      |

## Description

When `convert_to_usdc=true`, the proportion-based reward split uses integer division without rounding protection.

## Code Location

**File:** `instructions/decrease_liquidity.rs:406-410`

```
let reward_out_in_target_est = if total_other_in == 0 || reward_other_in == 0
{
    0u64
} else {
    let num = (total_out_in_target as u128)
        .checked_mul(reward_other_in as u128)
        .ok_or(LpDepositError::MathOverflow)?;
    let den = total_other_in as u128;
    (num / den) as u64 // Truncation
};
```

Precision loss accumulates over many transactions, consistently favoring the user (slightly less fee charged).

## Recommendation

Add rounding compensation:

```
let (quotient, remainder) = (num / den, num % den);
let reward_out = if remainder * 2 >= den { quotient + 1 } else { quotient }
as u64;
```

## Review Notes

Verify whether truncation consistently favors the user or the protocol. If it always favors the user (less fee charged), this is a slow fee leakage. Quantify the precision loss for typical pool sizes and fee percentages. Also verify whether the truncation in `(num / den) as u64` could cause issues when `num` exceeds `u64::MAX` (the cast from u128 to u64 silently truncates).

## Required Tests

1. **Precision loss quantification** -- Run a series of reward splits with varying `total_out_in_target` , `reward_other_in` , and `total_other_in` . Compare integer result with floating-point expected value. Quantify cumulative loss over 1000 operations.
2. **total\_other\_in near zero** -- Use a very small `total_other_in` to maximize the remainder. Verify result is reasonable.
3. **u128 to u64 truncation** -- Use large values that could cause `(num / den)` to exceed `u64::MAX` . Verify no silent truncation.
4. **Both tokens have zero reward** -- Verify correct behavior when `reward_other_in = 0` and `total_other_in = 0` .

---

## 6.9 LPH-009: Fixed 100-Unit Buffer in Min Output

|          |                 |
|----------|-----------------|
|          |                 |
| Severity | MEDIUM          |
| BVSS     | 4.5             |
| Status   | REQUIRES REVIEW |
| Category | Arithmetic      |

### Description

The minimum output calculation subtracts a fixed 100 units regardless of token decimals.

### Code Location

File: `state/utils.rs:86`

```
let out = out.checked_sub(100).unwrap_or(0);
```

For tokens with 0--2 decimals (or very low value per unit), 100 units could represent a significant amount.

### Recommendation

Use a percentage-based buffer:

```
let buffer = std::cmp::max(out / 10_000, 1); // 0.01% minimum
let out = out.saturating_sub(buffer);
```

## Review Notes

Verify the actual range of token decimals for pools in the whitelist. If all whitelisted pools use tokens with  $\geq 6$  decimals, the fixed 100-unit buffer is negligible. However, if the program could be used with tokens of 0--2 decimals in the future, this becomes significant. Also check whether `out < 100` resulting in `min_out = 0` could cause the swap CPI to accept any amount (including 0).

## Required Tests

1. **Token with 0 decimals** -- Call `calc_min_amount_out` for a swap involving a token with 0 decimals. Verify the 100-unit subtraction doesn't consume the entire output.
2. **out < 100 edge case** -- Verify behavior when `calc_min_amount_out` produces `out < 100`, resulting in `min_out = 0`. Confirm the swap CPI still behaves correctly.
3. **Token with 18 decimals** -- Verify the 100-unit buffer is negligible for high-decimal tokens.
4. **Large swap amount** -- Verify the fixed buffer is proportionally insignificant for large swaps.

## 6.10 LPH-010: Fee Calculation Precision Loss

|          |                 |
|----------|-----------------|
|          |                 |
| Severity | MEDIUM          |
| BVSS     | 4.0             |
| Status   | REQUIRES REVIEW |
| Category | Arithmetic      |

### Description

Multiple sequential divisions in `calc_min_amount_out` ( `utils.rs:50-77` ) cause accumulated precision loss through truncation at each step.

```
let out_after_fee = (out_before * fee_factor) / U256::from(1_000_000u128);
let out = (out_after_fee * slippage_factor) / U256::from(10_000u128);
```

Each division truncates, and the compound effect increases with the number of operations.

### Recommendation

Combine multiplications before division:

```
let out = (amount_in * price_q128 * fee_factor * slippage_factor)
  / (U256::from(1_000_000u128) * U256::from(10_000u128) << 64);
```

## Review Notes

Verify whether the precision loss in `calc_min_amount_out` compounds only at the min-output estimation level (slippage protection) or also affects actual fee/reward calculations. If it only affects `min_out` for swap CPI, the impact is a slightly looser slippage bound rather than actual fund loss. Also verify whether U256 intermediate results fit within 256 bits when all multiplications are combined.

## Required Tests

1. **Compare combined vs sequential division** -- For a set of representative inputs, compare the result of sequential `(a * b) / c * d) / e` vs combined `(a * b * d) / (c * e)` . Quantify the difference.
2. **Extreme fee\_rate and slippage values** -- Use maximum `fee_rate` (1\_000\_000) and maximum slippage (5000 bps). Verify `min_out` is still reasonable.
3. **Very small amount\_in** -- Use `amount_in = 1` . Verify `min_out` doesn't become negative or nonsensical.
4. **U256 overflow boundary** -- Use maximum `sqrt_price_x64` and `amount_in` to test if combined multiplication overflows U256.

---

### 6.11 LPH-011: init\_security\_config Has No Admin Gate

|          |                 |
|----------|-----------------|
|          |                 |
| Severity | MEDIUM          |
| BVSS     | 5.0             |
| Status   | REQUIRES REVIEW |
| Category | Governance      |

## Description

Anyone can call `init_security_config` if the PDA doesn't exist. The first signer becomes the authority.

## Code Location

**File:** `instructions/security_config.rs:51-66`

## Risk Scenarios

1. **Fresh deployment race:** Between deploy and init, an attacker front-runs initialization
2. **Post-close hijack:** After `close_security_config` , attacker re-initializes with their authority

## Recommendation

Hardcode an admin pubkey or require the program upgrade authority:

```
const ADMIN_KEY: Pubkey = pubkey!("...");
require_keys_eq!(ctx.accounts.authority.key(), ADMIN_KEY, ...);
```

## Review Notes

Verify the deployment procedure: is `init_security_config` called in the same transaction as program deployment? If so, front-running is impossible. Also verify whether the program is deployed with an upgrade authority -- if the program can be upgraded, the upgrade authority effectively controls the admin key (they could redeploy with a hardcoded key). Check if the `close_security_config` + re-`init_security_config` sequence is protected against authority hijack.

## Required Tests

1. **Non-admin calls `init_security_config` on fresh PDA** -- Deploy program, then have a non-admin account call `init_security_config` . Verify they become the authority.
2. **Re-init after close** -- Call `close_security_config` , then have a different account call `init_security_config` . Verify the new account becomes authority.
3. **`init_security_config` when PDA already exists** -- Attempt to call `init_security_config` when PDA is already initialized. Verify failure.
4. **Deployment + init in same transaction** -- Verify atomicity of deploy + init to prevent front-running.

---

## 6.12 LPH-012: Authority Key Not Rotatable

| Severity | MEDIUM          |
|----------|-----------------|
| BVSS     | 4.5             |
| Status   | REQUIRES REVIEW |
| Category | Governance      |

## Description

The `security_config.authority` is set once at initialization and cannot be changed. No `update_authority` instruction exists. If the admin key is compromised or lost, there is no recovery mechanism.

## Recommendation

Add an authority transfer instruction with two-step (propose + accept) pattern.

## Review Notes

Verify whether the program upgrade authority provides an implicit key rotation mechanism (redploy with new hardcoded admin). If the program is immutable (no upgrade authority), this finding is more severe. Also consider whether a multi-sig (e.g., Squads) is used as the authority address, which would provide recovery through multi-sig member rotation.

## Required Tests

1. **update\_security\_config with wrong authority** -- Call `update_security_config` with a signer that is NOT the stored authority. Verify failure.
2. **Verify authority field is immutable via update** -- Call `update_security_config` and attempt to change the `authority` field. Verify it remains unchanged.
3. **Authority persistence after update** -- Update pools/fee\_owners, then verify authority is still the original key.

---

### 6.13 LPH-013: Token-2022 Extension Checks Not Implemented

| Severity | MEDIUM          |
|----------|-----------------|
| BVSS     | 4.0             |
| Status   | REQUIRES REVIEW |
| Category | Token-2022      |

## Description

The error code `SecurityToken2022ForbiddenExtension` ( `errors.rs:79-80` ) is defined but never used. No code checks for dangerous Token-2022 extensions (PermanentDelegate, TransferHook, ConfidentialTransfer, NonTransferable).

The pool whitelist is the sole mitigation, relying on admin diligence when adding pools.

## Recommendation

Either implement extension checks in the entry phase:

```
// Check Token-2022 mint extensions
if ai.owner == &anchor_spl::token_2022::ID {
    let data = ai.data.borrow();
    let state = StateWithExtensions::<Mint>::unpack(&data)?;
    if state.get_extension::<PermanentDelegate>().is_ok() {
        return err!(LpDepositError::SecurityToken2022ForbiddenExtension);
    }
}
```

Or document clearly that the pool whitelist mechanism is the intentional control.

## Review Notes

Verify whether Raydium CLMM pools currently use any tokens with Token-2022 extensions. Check whether the admin pool whitelist review process includes Token-2022 extension inspection. Also verify whether the `parse_token2022_account` function in `security/mod.rs` correctly handles tokens with extensions (extension data could alter struct layout).

## Required Tests

1. **Whitelisted pool with PermanentDelegate token** -- Create a pool with a Token-2022 mint that has PermanentDelegate. Whitelist the pool. Verify whether the delegate can drain funds after a successful exit check.
2. **Whitelisted pool with TransferHook token** -- Create a pool with a TransferHook-enabled mint. Verify whether the hook program appears in the executable whitelist check.
3. **Token-2022 account parsing with extensions** -- Verify `parse_token2022_account` correctly extracts delegate and close\_authority from accounts with various extensions enabled.
4. **Non-Token-2022 pool unaffected** -- Verify that standard SPL Token pools are not affected by Token-2022 extension checks.

## 6.14 LPH-014: close\_security\_config Blocks All Operations

|          |                 |
|----------|-----------------|
|          |                 |
| Severity | MEDIUM          |
| BVSS     | 5.0             |
| Status   | REQUIRES REVIEW |
| Category | Governance      |

### Description

If the `security_config` PDA is closed, all business instructions immediately fail because `load_security_config()` returns an error when the PDA is missing. Users with existing positions cannot call `decrease_liquidity` to withdraw.

## Recommendation

- Document the operational procedure for security\_config management
- Consider adding a timelock before close takes effect
- Consider allowing emergency withdrawals even without security\_config

## Review Notes

Verify the operational runbook for security\_config management. Check whether the admin has a documented procedure for safely closing and re-initializing the config. Also verify whether users can directly interact with Raydium CLMM to withdraw their positions without going through this program (as a last-resort escape hatch).

## Required Tests

1. **Business instruction after close\_security\_config** -- Close the security\_config PDA, then call `decrease_liquidity` . Verify it fails with `SecurityPoolWhitelistMissing` .
2. **Re-init after close restores operations** -- Close the PDA, re-initialize with the same pools, then call `decrease_liquidity` . Verify it succeeds.
3. **close\_security\_config with wrong authority** -- Verify only the authority can close.
4. **close\_security\_config receiver gets rent** -- Verify the receiver account receives the rent refund.

---

## 6.15 LPH-015: No Events on Security Config Changes

|          |                 |
|----------|-----------------|
|          |                 |
| Severity | LOW             |
| BVSS     | 2.5             |
| Status   | REQUIRES REVIEW |
| Category | Observability   |

## Description

`init_security_config` , `update_security_config` , and `close_security_config` emit no events. Configuration changes are silent on-chain, making monitoring and audit trail impossible.

## Recommendation

Emit events on all configuration changes:

```
emit!(SecurityConfigUpdated {
    authority: ctx.accounts.authority.key(),
    pools: pools.clone(),
    fee_owners: fee_owners.clone(),
});
```

## Review Notes

Verify whether any off-chain monitoring currently watches for security\_config account changes via `getAccountInfo` polling. Even without events, account data changes are detectable via Solana's websocket subscriptions. Events would provide a cleaner audit trail.

## Required Tests

1. **Verify no events emitted on init** -- Call `init_security_config` and check transaction logs for emitted events. Confirm none exist.
2. **Verify no events emitted on update** -- Call `update_security_config` and check transaction logs. Confirm none exist.
3. **After fix, verify events contain correct data** -- After implementing events, verify they contain the correct authority, pools, and fee\_owners.

## 6.16 LPH-016: `.unwrap()` Instead of Error Propagation

|          |                 |
|----------|-----------------|
|          |                 |
| Severity | LOW             |
| BVSS     | 2.0             |
| Status   | REQUIRES REVIEW |
| Category | Code Quality    |

### Description

Two locations use `.unwrap()` on `try_into()` which could panic:

| Location                  | Code                                       |
|---------------------------|--------------------------------------------|
| <code>utils.rs:122</code> | <code>liquidity.try_into().unwrap()</code> |
| <code>utils.rs:159</code> | <code>liquidity.try_into().unwrap()</code> |

If `liquidity` (u128) exceeds `i128::MAX`, this panics and aborts the transaction.

## Recommendation

```
liquidity.try_into().map_err(|_| error!(LpDepositError::MathOverflow))?
```

## Review Notes

Verify the maximum possible value of `liquidity` in practice. If Raydium CLMM liquidity values never exceed `i128::MAX` (which is likely given pool sizes), the panic is theoretically unreachable. However, defensive programming requires proper error handling regardless.

## Required Tests

1. `liquidity = i128::MAX + 1` -- Call the function with `liquidity = i128::MAX as u128 + 1`. Verify the transaction fails gracefully (not panic).
2. `liquidity = 0` -- Verify correct behavior with zero liquidity.
3. `liquidity = u128::MAX` -- Verify the function handles maximum u128 value.
4. **Normal liquidity values** -- Verify typical pool liquidity values work correctly.

## 6.17 LPH-017: Direct Multiplication Without `checked_mul`

| Severity | LOW             |
|----------|-----------------|
| BVSS     | 1.5             |
| Status   | REQUIRES REVIEW |
| Category | Arithmetic      |

## Description

Direct multiplication `i * 32` in `security_config` byte offset calculations:

| Location                         | Code                                    |
|----------------------------------|-----------------------------------------|
| <code>security/mod.rs:170</code> | <code>start + i * 32</code>             |
| <code>security/mod.rs:224</code> | <code>pools_start + pools_n * 32</code> |
| <code>security/mod.rs:228</code> | <code>fee_start + i * 32</code>         |

These are bounded by `MAX_ALLOWED_POOLS=40` and `MAX_FEE_OWNERS=6`, making overflow impossible in practice, but `checked_mul` would be more defensive.

## Recommendation

Use `checked_mul` for consistency with the rest of the codebase.

## Review Notes

Verify the bounds: `MAX_ALLOWED_POOLS = 40` means maximum `i = 39`, so `i * 32 = 1248` which is well within `usize` range on all platforms. This finding is purely a code quality / defense-in-depth concern, not a practical vulnerability.

## Required Tests

1. **security\_config with MAX\_ALLOWED\_POOLS pools** -- Initialize `security_config` with exactly 40 pools. Verify all pools are correctly read from the byte offsets.
2. **security\_config with MAX\_FEE\_OWNERS fee\_owners** -- Initialize with 6 `fee_owners`. Verify all are correctly read.
3. **Pool lookup at last index** -- Verify the pool at index 39 (the last) is correctly found by `security_config_contains`.

---

### 6.18 LPH-018: `unwrap_or(0)` Pattern Masks Errors

|          |                 |
|----------|-----------------|
|          |                 |
| Severity | LOW             |
| BVSS     | 1.0             |
| Status   | REQUIRES REVIEW |
| Category | Code Quality    |

## Description

Multiple uses of `.unwrap_or(0)` silently convert errors to zero:

- `zap_common.rs` : 8 occurrences
- `decrease_liquidity.rs` : 4 occurrences

While intentional for edge cases, this could mask real calculation errors.

## Recommendation

Add logging when the fallback triggers:

```
.unwrap_or_else(|| { msg!("WARN: calculation fallback to 0"); 0 })
```

## Review Notes

Review each `unwrap_or(0)` usage site to determine if the fallback is truly safe. Some locations (e.g., `checked_sub` for balance differences) may be intentional safety nets for race conditions, while others may mask real bugs. Categorize each occurrence as "intentional fallback" or "needs investigation".

## Required Tests

1. **Trigger `unwrap_or(0)` in balance calculation** -- Create a scenario where `checked_sub` on balance produces `None` (e.g., account balance changes between reads). Verify the `unwrap_or(0)` path produces correct downstream behavior.
2. **`unwrap_or(0)` in fee calculation** -- Verify that when fee calculation falls back to 0, no unexpected side effects occur (e.g., division by zero in subsequent calculations).
3. **Enumerate all `unwrap_or(0)` sites** -- Create a test that exercises each `unwrap_or(0)` call site to verify the fallback behavior is acceptable in each context.

### 6.19 LPH-019: Caller-Controlled `fee_percent` Allows Fee Bypass

|          |                 |
|----------|-----------------|
|          |                 |
| Severity | HIGH            |
| BVSS     | 7.5             |
| Status   | REQUIRES REVIEW |
| Category | Business Logic  |

## Description

The `fee_percent` parameter is passed directly by the caller as an instruction argument with only an upper bound check ( `<= 10_000` ). There is no minimum fee enforcement. A caller can set `fee_percent = 0` to withdraw all rewards/fees without paying the integrator fee.

## Code Location

File: `instructions/decrease_liquidity.rs:202-207`

```
require_log!(
    fee_percent <= 10_000,
    LpDepositError::InvalidFeePercent,
    "fee_percent={}",
    fee_percent
);
```

## Impact

The protocol's entire fee revenue depends on the caller providing a non-zero `fee_percent`. If this program is designed as an integrator/aggregator that collects fees, any caller can bypass fees entirely by passing `fee_percent = 0`.

**Note:** This may be **by design** if the calling frontend/SDK always sets the correct `fee_percent` and the trust model is "the signer is the integrator's own backend." If the program is intended to be called by arbitrary users, this is a HIGH vulnerability.

## BVSS Score

| Metric            | Value           |
|-------------------|-----------------|
| Attack Origin     | Arbitrary (1.0) |
| Attack Cost       | Low (1.0)       |
| Attack Complexity | Low (1.0)       |
| Impact            | High (0.75)     |
| <b>BVSS Score</b> | <b>7.5</b>      |

## Recommendation

If mandatory fees are intended, add a minimum fee check or store the fee rate in the SecurityConfig PDA:

```
const MIN_FEE_BPS: u64 = 10; // Example: 0.1% minimum
require!(fee_percent >= MIN_FEE_BPS, LpDepositError::InvalidFeePercent);
```

Alternatively, store the canonical fee rate in `security_config` so callers cannot influence it:

```
let fee_percent = security_config.fee_bps; // Admin-controlled, not caller-
controlled
```

## Review Notes

This finding depends on the trust model. If the program is only called by the protocol's own backend (which always sets the correct `fee_percent`), this is by design. If arbitrary users can call this instruction directly, fee bypass is trivial. The development team should clarify the intended trust model and document it.

## Required Tests

1. **fee\_percent = 0 bypasses all fees** -- Call `decrease_liquidity` with `fee_percent = 0` . Verify no fees are collected and all rewards go to the user.
2. **fee\_percent = 1 (minimum)** -- Verify fee calculation works correctly for very small fee percentages.
3. **fee\_percent = 10000 (100%)** -- Verify all reward goes to fee\_owner and user receives only principal.
4. **Minimum fee enforcement (after fix)** -- After implementing a minimum, verify `fee_percent < MIN_FEE_BPS` is rejected.

## 6.20 LPH-020: Fee Rounding Exploitable via Micro-Transactions

|          |                 |
|----------|-----------------|
|          |                 |
| Severity | LOW             |
| BVSS     | 2.0             |
| Status   | REQUIRES REVIEW |
| Category | Arithmetic      |

### Description

Fee calculation uses `checked_mul(fee_percent).checked_div(10_000)` which rounds down. For amounts where `reward * fee_percent < 10_000` , the fee becomes 0. An attacker could theoretically split a large withdrawal into many micro-transactions to avoid fees entirely.

### Code Location

**File:** `instructions/decrease_liquidity.rs:420-424, 516-524`

### Impact

The gas cost of multiple transactions on Solana (~5000 lamports each) makes this uneconomical for typical fee rates. However, for very low fee rates (e.g., `fee_percent = 1` = 0.01%), the threshold for zero-fee withdrawals is higher, and the attack becomes more practical.

### Recommendation

Add a minimum fee floor when reward is non-zero:

```
let fee =  
reward.checked_mul(fee_percent).unwrap().checked_div(10_000).unwrap();  
let fee = if fee == 0 && reward > 0 && fee_percent > 0 { 1 } else { fee };
```

## Review Notes

Quantify the economic viability: for a typical `fee_percent = 100` (1%), a reward of less than 100 base units would produce `fee = 0`. For tokens with 6+ decimals, 100 base units is negligible (\$0.0001 for USDC). This is likely uneconomical to exploit in practice.

## Required Tests

1. **Reward = 99 with `fee_percent = 100`** -- Verify fee rounds to 0.
2. **Reward = 100 with `fee_percent = 100`** -- Verify fee = 1 (boundary).
3. **Multiple micro-withdrawals vs single large withdrawal** -- Compare total fees paid across 100 micro-withdrawals vs 1 equivalent large withdrawal.

---

### 6.21 LPH-021: Missing overflow-checks in Release Profile

|          |                 |
|----------|-----------------|
|          |                 |
| Severity | LOW             |
| BVSS     | 1.5             |
| Status   | REQUIRES REVIEW |
| Category | Build Config    |

## Description

The program uses `checked_*` arithmetic throughout, which is good practice. However, the `cargo.toml` does not explicitly set `overflow-checks = true` in the release profile within the program crate. Without this flag, any arithmetic using native operators ( `+` , `-` , `*` ) will silently wrap on overflow in release builds.

## Impact

The program relies on explicit `checked_*` calls rather than implicit overflow detection, which is generally more reliable and not dependent on the build profile. The risk is that a future code change might introduce native arithmetic without `checked_*` , which would silently overflow in release builds but panic in debug builds, creating a testing gap.

## Recommendation

Add to the program's `Cargo.toml` :

```
[profile.release]
overflow-checks = true
```

## Review Notes

Verify all arithmetic in the codebase uses `checked_*` operations. If there are any native `+`, `-`, `*` operations on u64/u128/i128 values (other than in constants or compile-time expressions), this finding is elevated in severity. LPH-017 (direct multiplication in byte offsets) is a related example.

## Required Tests

1. **Verify overflow-checks flag in Cargo.toml** -- Check the release profile for `overflow-checks = true`.
  2. **Audit all native arithmetic** -- Enumerate all uses of `+`, `-`, `*` on integer types outside of `checked_*` wrappers.
-

## 7. SlowMist Best Practices Compliance

### 7.1 Common Pitfalls

| #  | Category                                 | Status         | Notes                                                                                                            |
|----|------------------------------------------|----------------|------------------------------------------------------------------------------------------------------------------|
| 1  | Integer overflow/<br>underflow           | MOSTLY<br>SAFE | <code>checked_*</code> used extensively; U256 in <code>utils.rs:50-77</code> uses native operators (see LPH-017) |
| 2  | Loss of precision                        | SAFE           | No <code>try_round_u64()</code> usage; floor semantics used                                                      |
| 3  | Inaccurate calculation<br>(saturating_*) | ISSUE          | <code>decrease_liquidity.rs:288-289</code> uses <code>saturating_sub</code> (see LPH-004)                        |
| 4  | Division by zero                         | SAFE           | <code>utils.rs:51</code> checks <code>!price_q128.is_zero()</code>                                               |
| 5  | Error not handled                        | MOSTLY<br>SAFE | <code>.unwrap()</code> at <code>utils.rs:122,159</code> (see LPH-016)                                            |
| 6  | Caller permission<br>check               | SAFE<br>(NOTE) | Anyone can init <code>security_config</code> (see LPH-011)                                                       |
| 7  | Account signer check                     | SAFE           | All handlers use <code>Signer&lt;'info&gt;</code>                                                                |
| 8  | Account writable check                   | SAFE           | Properly annotated with <code>#[account(mut)]</code>                                                             |
| 9  | Account owner/<br>program ID check       | SAFE           | Security layer validates owners                                                                                  |
| 10 | Account initialized<br>check             | SAFE           | Anchor <code>init</code> prevents re-initialization                                                              |
| 11 | PDA substitution check                   | SAFE           | Uses <code>find_program_address</code>                                                                           |
| 12 | System account check                     | SAFE           | Anchor typed wrappers                                                                                            |
| 13 | Lamport check                            | SAFE           | Exit check validates lamport leakage                                                                             |
| 14 | Pyth oracle check                        | N/A            | No oracle usage                                                                                                  |
| 15 | Timely state reset                       | SAFE           | Exit checks enforce no <code>delegate/close_authority</code>                                                     |

## 7.2 Anchor Framework Attacks

| #  | Category                   | Status                 | Notes                                                                                          |
|----|----------------------------|------------------------|------------------------------------------------------------------------------------------------|
| 16 | Signer authorization       | SAFE                   | <code>Signer&lt;'info&gt;</code> on all entry points                                           |
| 17 | Account data matching      | SAFE                   | Token mint/authority Anchor constraints                                                        |
| 18 | Owner checks               | SAFE                   | <code>InterfaceAccount&lt;TokenAccount&gt;</code> + security layer                             |
| 19 | Type cosplay               | SAFE                   | Anchor discriminators prevent confusion                                                        |
| 20 | Check initialize           | SAFE                   | <code>init</code> constraint with PDA seeds                                                    |
| 21 | Arbitrary CPI              | SAFE                   | <code>#[account(address = raydium_amm_v3::ID)]</code> + remaining accounts owner check         |
| 22 | Duplicate mutable accounts | <b>POTENTIAL ISSUE</b> | <code>fee_owner == recipient</code> not prevented;<br><code>recipient == signer</code> handled |
| 23 | Bump seed canonicalization | SAFE                   | Anchor automatic bump                                                                          |
| 24 | PDA sharing                | SAFE                   | Singleton PDA with unique seeds                                                                |
| 25 | Closing accounts           | SAFE                   | Anchor <code>close</code> constraint zeroes data                                               |
| 26 | Sysvar address checking    | SAFE                   | Anchor typed <code>Sysvar&lt;'info, Rent&gt;</code>                                            |
| 27 | Account reloading          | SAFE                   | <code>.reload()</code> after all CPIs                                                          |

### 7.3 Additional Findings from SlowMist Review

#### UncheckedAccount Usage

| Location                               | Account                           | Validation          | Risk           |
|----------------------------------------|-----------------------------------|---------------------|----------------|
| <code>swap_and_deposit.rs:38</code>    | <code>recipient</code>            | Security layer only | HIGH (LPH-002) |
| <code>swap_and_deposit.rs:92</code>    | <code>position_nft_account</code> | CPI validates       | LOW            |
| <code>decrease_liquidity.rs:38</code>  | <code>recipient</code>            | Security layer only | HIGH (LPH-002) |
| <code>decrease_liquidity.rs:161</code> | <code>position_nft_account</code> | <b>NONE</b>         | HIGH (LPH-006) |
| <code>security_config.rs:96</code>     | <code>receiver</code>             | None (rent refund)  | LOW            |

#### Duplicate Mutable Accounts Gap

The code handles `recipient == signer` case at `decrease_liquidity.rs:607` :

```
let recipient_is_signer = ctx.accounts.recipient.key() ==  
ctx.accounts.signer.key();
```

But there is no check preventing `fee_owner == recipient` , which could allow fee bypass if the same address serves as both fee collector and fund recipient.

---

## 8. Handler-Level Security Assessment

### 8.1 Permission Matrix

| Handler                | Signer              | Admin Only | Pool Whitelist | Fee Owner Whitelist  | Security Sandwich |
|------------------------|---------------------|------------|----------------|----------------------|-------------------|
| swap_and_deposit       | Yes                 | No         | Yes            | Yes (token accounts) | Yes               |
| increase_liquidity     | Yes + NFT Owner     | No         | Yes            | Yes (token accounts) | Yes               |
| decrease_liquidity     | Yes                 | No         | Yes            | Yes (token accounts) | Yes               |
| init_security_config   | Yes (becomes admin) | N/A        | N/A            | N/A                  | <b>No</b>         |
| update_security_config | Yes                 | Yes        | N/A            | N/A                  | No                |
| close_security_config  | Yes                 | Yes        | N/A            | N/A                  | No                |

### 8.2 swap\_and\_deposit Account Constraints

**Source:** [instructions/swap\\_and\\_deposit.rs:30-176](#)

| Account               | Type                           | Key Constraints                                |
|-----------------------|--------------------------------|------------------------------------------------|
| raydium_clmm_program  | Program<AmmV3>                 | address = raydium_amm_v3::ID                   |
| signer                | Signer<'info>                  | Must sign transaction                          |
| recipient             | UncheckedAccount               | <b>No on-chain validation</b>                  |
| signer_token0_account | InterfaceAccount<TokenAccount> | mint = vault_0.mint ,<br>authority = signer    |
| signer_token1_account | InterfaceAccount<TokenAccount> | mint = vault_1.mint ,<br>authority = signer    |
| fee_owner             | SystemAccount                  | <b>No address constraint</b>                   |
| fee_token0_account    | InterfaceAccount<TokenAccount> | mint = vault_0.mint ,<br>authority = fee_owner |
| fee_token1_account    | InterfaceAccount<TokenAccount> | mint = vault_1.mint ,<br>authority = fee_owner |
| amm_config            | UncheckedAccount               | address =<br>pool_state.load()?.amm_config     |
| pool_state            | AccountLoader<PoolState>       | Pool whitelist validated by security layer     |
| token_vault_0         | InterfaceAccount<TokenAccount> | key() ==<br>pool_state.token_vault_0           |
| token_vault_1         | InterfaceAccount<TokenAccount> | key() ==<br>pool_state.token_vault_1           |
| security_config       | UncheckedAccount               | PDA seeds<br>[SECURITY_CONFIG_SEED]            |
| position_nft_mint     | Signer                         | Must sign (keypair generated client-side)      |
| position_nft_account  | UncheckedAccount               | Address checked via ATA derivation             |

**Remaining Accounts:** [swap\_remaining] + [separator] + [action\_remaining]

- swap\_remaining : length <= 32, all owners == raydium\_clmm\_program
- Separator: crate::ID , must appear exactly once

### 8.3 decrease\_liquidity Account Constraints

Source: `instructions/decrease_liquidity.rs:29-189`

| Account                               | Type                                              | Key Constraints                                          |
|---------------------------------------|---------------------------------------------------|----------------------------------------------------------|
| <code>recipient_token0_account</code> | <code>InterfaceAccount&lt;TokenAccount&gt;</code> | <code>mint = vault_0.mint , authority = recipient</code> |
| <code>recipient_token1_account</code> | <code>InterfaceAccount&lt;TokenAccount&gt;</code> | <code>mint = vault_1.mint , authority = recipient</code> |
| <code>fee_token0_account</code>       | <code>InterfaceAccount&lt;TokenAccount&gt;</code> | <code>mint = vault_0.mint , authority = fee_owner</code> |
| <code>fee_token1_account</code>       | <code>InterfaceAccount&lt;TokenAccount&gt;</code> | <code>mint = vault_1.mint , authority = fee_owner</code> |
| <code>personal_position</code>        | <code>Account&lt;PersonalPosition&gt;</code>      | <code>pool_id == pool_state.key()</code>                 |
| <code>position_nft_account</code>     | <code>UncheckedAccount</code>                     | <b>No constraints</b><br>(LPH-006)                       |

#### Fee Calculation:

- **Principal:** estimated via `calculate_principal_amounts_for_liquidity()` based on current tick/price
- **Reward:** `balance_increase - principal` (using `saturating_sub` )
- **Fee:** `reward * fee_percent / 10_000` (only on rewards)

### 8.4 increase\_liquidity Account Constraints

Source: `instructions/increase_liquidity.rs:32-172`

Key differences from `swap_and_deposit` :

- `recipient = signer` always ( `lib.rs:141` ) -- no separate recipient
- `position_nft_account` : **Properly validated** ( `mint == nft_mint , amount == 1 , authority = signer` )
- Uses `increase_liquidity_v2` CPI instead of `open_position`

## 8.5 Security Config Admin Constraints

| Instruction                         | Authority Check                                           | Validation                                                                                                       |
|-------------------------------------|-----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <code>init_security_config</code>   | Signer becomes authority                                  | <code>!pools.is_empty()</code> ,<br><code>pools.len() &lt;= 40</code> ,<br><code>fee_owners.len() &lt;= 6</code> |
| <code>update_security_config</code> | <code>security_config.authority == authority.key()</code> | Same validations as init                                                                                         |
| <code>close_security_config</code>  | <code>security_config.authority == authority.key()</code> | <code>close = receiver</code>                                                                                    |

## 9. Positive Security Observations

The program demonstrates several strong security practices:

1. **Security Sandwich Macro** -- All business instructions wrapped by `secure_entrypoint!` for consistent entry/exit validation; this is a well-designed architectural pattern
2. **Pool Whitelist via PDA** -- `security_config` enforces that only admin-approved pools can be used
3. **Executable Program Whitelist** -- Hardcoded `ALLOWED_EXECUTABLE_PROGRAMS` (7 programs) prevents arbitrary CPI targets
4. **Token Authority Exit Check** -- Comprehensive validation that delegate, close\_authority, and authority haven't changed
5. **Account Reload After CPI** -- Proper `.reload()` calls after every Raydium CPI  
( `zap_common.rs:280-281` , `decrease_liquidity.rs:268-270` ,  
`decrease_liquidity.rs:389-390` , `decrease_liquidity.rs:592-593` )
6. **Checked Arithmetic** -- 54+ instances of `checked_add` / `checked_sub` / `checked_mul` / `checked_div` across the codebase
7. **Raydium Program Validation** -- Hardcoded  
`#[account(address = raydium_amm_v3::ID)]` on all CPI target accounts
8. **Remaining Accounts Owner Validation** -- All swap tick arrays validated for Raydium ownership ( `zap_common.rs:637-644` )
9. **SOL Leakage Prevention** -- Program-owned accounts checked for excess lamports at exit
10. **Anchor Framework** -- Proper use of typed accounts, discriminators, PDA seeds, and constraints
11. **No-Heap Logging** -- `require_log!` macro uses `msg!` without heap allocation
12. **New Account Owner Validation** -- Newly initialized accounts must be owned by whitelisted programs

## 10. Operational Security Recommendations

---

Based on the [Solana Application Security Roadmap](#).

### 10.1 Pre-Deployment

| Item                    | Status                 | Recommendation                                                    |
|-------------------------|------------------------|-------------------------------------------------------------------|
| Multi-sig for admin key | <b>Not implemented</b> | Use <a href="#">Squads Multisig</a> for security_config authority |
| Fuzz testing            | Unknown                | Implement with <a href="#">Trident</a>                            |
| Formal verification     | Unknown                | Consider <a href="#">Certora Solana Prover</a>                    |
| Automated SAST          | Partial                | Integrate SolanaGuard or sec3 into CI                             |

### 10.2 Post-Deployment

| Item                              | Status              | Recommendation                                        |
|-----------------------------------|---------------------|-------------------------------------------------------|
| Bug bounty program                | Unknown             | Launch on Immunefi or similar                         |
| Incident Response Plan            | Unknown             | Develop IRP following <a href="#">Crisis Handbook</a> |
| On-chain monitoring               | Unknown             | Integrate <a href="#">Watchtower by Sec3</a>          |
| Security config change monitoring | <b>Not possible</b> | Events not emitted (see LPH-015)                      |
| Admin key security                | Unknown             | Use hardware keys for production                      |

---

## 11. Recommendations Summary

---

### 11.1 By Priority

#### Immediate (Before Mainnet)

| Priority | Action                                                             | Finding |
|----------|--------------------------------------------------------------------|---------|
| CRITICAL | Add fee_owner whitelist validation in handler before fee transfers | LPH-001 |
| CRITICAL | Require recipient as Signer or validate on-chain authorization     | LPH-002 |

### High (Before Audit Completion)

| Priority | Action                                                                    | Finding |
|----------|---------------------------------------------------------------------------|---------|
| HIGH     | Reject pre-existing delegates/close_authority at entry                    | LPH-003 |
| HIGH     | Replace <code>saturating_sub</code> with explicit handling in reward calc | LPH-004 |
| HIGH     | Store and verify account keys in exit check snapshot                      | LPH-005 |
| HIGH     | Unify position_nft_account validation across all instructions             | LPH-006 |
| HIGH     | Verify recipient exists in accounts array in security layer               | LPH-007 |
| HIGH     | Add minimum fee_percent validation or store fee rate in SecurityConfig    | LPH-019 |

### Medium (Before Production Scale)

| Priority | Action                                                       | Finding |
|----------|--------------------------------------------------------------|---------|
| MEDIUM   | Add rounding compensation to reward split division           | LPH-008 |
| MEDIUM   | Use percentage-based buffer instead of fixed 100-unit        | LPH-009 |
| MEDIUM   | Reduce precision loss in fee calculation                     | LPH-010 |
| MEDIUM   | Protect init_security_config against front-running           | LPH-011 |
| MEDIUM   | Add authority rotation mechanism                             | LPH-012 |
| MEDIUM   | Implement Token-2022 extension checks or document mitigation | LPH-013 |
| MEDIUM   | Add timelock or safeguard for close_security_config          | LPH-014 |

## Low (Improvement)

| Priority | Action                                                      | Finding |
|----------|-------------------------------------------------------------|---------|
| LOW      | Emit events on security config changes                      | LPH-015 |
| LOW      | Replace <code>.unwrap()</code> with <code>?</code> operator | LPH-016 |
| LOW      | Use <code>checked_mul</code> in byte offset calculations    | LPH-017 |
| LOW      | Add logging to <code>unwrap_or(0)</code> fallback paths     | LPH-018 |
| LOW      | Add minimum fee floor to prevent rounding to zero           | LPH-020 |
| LOW      | Set <code>overflow-checks = true</code> in release profile  | LPH-021 |

## 11.2 By Category

| Category           | Findings                                                               | Actions                                                                       |
|--------------------|------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Account Validation | LPH-001, LPH-002, LPH-006                                              | Add on-chain validation for fee_owner and recipient; unify NFT account checks |
| Security Layer     | LPH-003, LPH-005, LPH-007                                              | Strengthen entry validation and exit re-verification                          |
| Arithmetic         | LPH-004, LPH-008, LPH-009, LPH-010, LPH-016, LPH-017, LPH-018, LPH-020 | Replace saturating_sub, improve precision, replace unwrap, fee floor          |
| Business Logic     | LPH-019                                                                | Add minimum fee_percent or store fee rate in SecurityConfig                   |
| Governance         | LPH-011, LPH-012, LPH-014, LPH-015                                     | Admin gate, key rotation, close safeguard, events                             |
| Token-2022         | LPH-013                                                                | Extension checks or documented pool whitelist reliance                        |
| Build Config       | LPH-021                                                                | Set <code>overflow-checks = true</code> in release profile                    |

## 12. Error Code Reference

Source: `state/errors.rs:1-113`

| Offset | Code | Name                                  | Description                                   |
|--------|------|---------------------------------------|-----------------------------------------------|
| 4000   | 4000 | MathOverflow                          | Arithmetic overflow                           |
| 4001   | 4001 | InvalidFeePercent                     | Fee > 100%                                    |
| 4002   | 4002 | InvalidSlippage                       | Slippage > 50%                                |
| 4003   | 4003 | InvalidDepositMint                    | return_mint/swap_to_token_mint not pool token |
| 4004   | 4004 | InvalidDepositAmount                  | Invalid deposit amount                        |
| 4005   | 4005 | InsufficientBalance                   | Insufficient token balance                    |
| 4006   | 4006 | OutOfRangeAtExecution                 | Pool price out of range at execution          |
| 4007   | 4007 | InvalidPositionNftOwner               | Invalid position NFT owner                    |
| 4008   | 4008 | InvalidPositionNftAccount             | Position NFT ATA mismatch                     |
| 4009   | 4009 | InvalidFeeTokenAccount                | Fee ATA validation failed                     |
| 4010   | 4010 | InvalidFeeOwner                       | Fee owner not in whitelist                    |
| 4011   | 4011 | InvalidRemainingAccounts              | Remaining accounts structure invalid          |
| 4012   | 4012 | EventLogSerializeFailed               | Event serialization failed                    |
| 4013   | 4013 | InvalidSqrtPrice                      | Invalid sqrt price input                      |
| 4014   | 4014 | InvalidTickRange                      | Invalid tick range                            |
| 4015   | 4015 | NoBalanceChange                       | No claimable rewards or principal             |
| 4016   | 4016 | SecurityConfigPoolsEmpty              | Pools list empty                              |
| 4017   | 4017 | SecuritySeparatorInvalid              | Remaining accounts separator invalid          |
| 4018   | 4018 | SecurityUnauthorizedExecutableProgram | Unauthorized executable in accounts           |
| 4019   | 4019 | SecurityPoolNotAllowed                | Pool not in whitelist                         |
| 4020   | 4020 | SecurityPoolWhitelistMissing          | SecurityConfig PDA not provided               |
| 4021   | 4021 | SecurityPoolWhitelistInvalid          | Invalid SecurityConfig PDA                    |
| 4022   | 4022 | SecurityConfigAdminUnauthorized       | Not admin                                     |
| 4023   | 4023 | SecurityToken2022ForbiddenExtension   | Forbidden Token-2022 extension                |

| Offset | Code | Name                                    | Description                     |
|--------|------|-----------------------------------------|---------------------------------|
| 4024   | 4024 | SecurityAccountSetChanged               | Account set modified            |
| 4025   | 4025 | SecurityProgramLamportsLeaked           | SOL leaked to program account   |
| 4026   | 4026 | SecurityTokenAccountCorrupted           | Token account corrupted at exit |
| 4027   | 4027 | SecurityTokenAuthorityChanged           | Token authority modified        |
| 4028   | 4028 | SecurityTokenDelegateNotAllowed         | Delegate set (forbidden)        |
| 4029   | 4029 | SecurityTokenCloseAuthorityNotAllowed   | Close authority set (forbidden) |
| 4030   | 4030 | SecurityDisallowedAccountOwner          | Invalid new account owner       |
| 4031   | 4031 | SecurityNewTokenAccountAuthorityInvalid | Invalid new token authority     |
| 4032   | 4032 | SecurityNonWhitelistTokenAccount        | Non-whitelisted token account   |

## Appendix A: Files Analyzed

| File                                            | Lines | Purpose                                                  |
|-------------------------------------------------|-------|----------------------------------------------------------|
| <code>lib.rs</code>                             | 182   | Program entry with <code>secure_entrypoint!</code> macro |
| <code>instructions/swap_and_deposit.rs</code>   | 308   | Swap + open position                                     |
| <code>instructions/increase_liquidity.rs</code> | 273   | Add to existing position                                 |
| <code>instructions/decrease_liquidity.rs</code> | 818   | Remove liquidity + fee extraction                        |
| <code>instructions/zap_common.rs</code>         | 911   | Shared swap/liquidity helpers                            |
| <code>instructions/security_config.rs</code>    | 97    | Security config PDA management                           |
| <code>security/mod.rs</code>                    | 528   | Core security layer (entry/exit checks)                  |
| <code>state/security_config.rs</code>           | 33    | SecurityConfig struct definition                         |
| <code>state/errors.rs</code>                    | 113   | Error code definitions                                   |
| <code>state/events.rs</code>                    | 78    | Event definitions                                        |
| <code>state/consts.rs</code>                    | 32    | Hardcoded whitelists                                     |
| <code>state/utils.rs</code>                     | 176   | Math utilities                                           |

## Appendix B: Security Invariants Summary

The `secure_entrypoint!` macro enforces 10 security invariants:

| #  | Invariant                                                      | Phase | Verification                                   |
|----|----------------------------------------------------------------|-------|------------------------------------------------|
| 1  | Only whitelisted pools can be interacted with                  | Entry | <code>security_config_contains()</code>        |
| 2  | Only whitelisted executable programs in account set            | Entry | <code>ALLOWED_EXECUTABLE_PROGRAMS</code> check |
| 3  | Only whitelisted token account authorities                     | Entry | signer/recipient/fee_owner/pool vault          |
| 4  | <code>remaining_accounts</code> separator appears at most once | Entry | Count check                                    |
| 5  | Program-owned accounts do not accumulate excess SOL            | Exit  | <code>lamports &lt;= rent_exempt_min</code>    |
| 6  | Signer's token account ownership cannot be transferred         | Exit  | <code>cur.owner == signer</code>               |
| 7  | Signer's token accounts must not have delegate                 | Exit  | <code>cur.delegate.is_none()</code>            |
| 8  | Signer's token accounts must not have close_authority          | Exit  | <code>cur.close_authority.is_none()</code>     |
| 9  | Newly created accounts owned by whitelisted programs           | Exit  | <code>ALLOWED_ACCOUNT_OWNERS</code> check      |
| 10 | Newly created token accounts owned by signer/recipient         | Exit  | <code>is_allowed_authority()</code>            |

---

## Appendix C: Hardcoded Whitelists

---

Source: `state/consts.rs`

## ALLOWED\_EXECUTABLE\_PROGRAMS

| Program                  | Pubkey                                                    |
|--------------------------|-----------------------------------------------------------|
| LP Handler (self)        | <code>crate::ID</code>                                    |
| System Program           | <code>11111111111111111111111111111111</code>             |
| SPL Token                | <code>TokenkegQfeZyiNwAJbNbGKPFXCWuBvf9Ss623VQ5DA</code>  |
| SPL Token-2022           | <code>TokenzQdBNbLqP5VEhdkAS6EPFLC1PHnBqCXEpPxuEb</code>  |
| Associated Token Account | <code>ATokenGPvbdGVxr1b2hvZbsiqW5xWH25efTNsLJA8knL</code> |
| SPL Memo                 | <code>MemoSq4gqABAXKb96qnH8TysNcWxMyWcQXgDLGmfCHr</code>  |
| Raydium AMM v3           | <code>CAMMCzo5YL8w4VFF8KVHrK22GGUsp5VTaW7grrKgrWqK</code> |

## ALLOWED\_ACCOUNT\_OWNERS

| Program                  |
|--------------------------|
| System Program           |
| SPL Token                |
| SPL Token-2022           |
| Associated Token Account |
| Raydium AMM v3           |

## Appendix D: State Structures

### SecurityConfig ( `state/security_config.rs:12-32` )

```
#[account]
pub struct SecurityConfig {
    pub authority: Pubkey,          // Admin (32 bytes)
    pub pools: Vec<Pubkey>,        // Max 40 (4 + 32*40 bytes)
    pub fee_owners: Vec<Pubkey>,   // Max 6 (4 + 32*6 bytes)
}
// Total SIZE: 8 + 32 + 4 + 32*40 + 4 + 32*6 = 1,508 bytes
// PDA Seeds: [b"security_config"]
```

SecuritySnapshot ( security/mod.rs:70-80 )

```
pub struct SecuritySnapshot<'info> {
    pub signer_token_accounts: Vec<SignerTokenAccountBefore<'info>>,
    pub uninitialized_indices: Vec<usize>,
}
```

SecurityPolicy ( security/mod.rs:9-22 )

```
pub struct SecurityPolicy {
    pub forbid_delegate: bool,          // Default: true
    pub forbid_close_authority: bool,  // Default: true
}
```

Appendix E: Automated Scanner False Positive Summary

| Pattern                        | Count | Reason False Positive                              |
|--------------------------------|-------|----------------------------------------------------|
| SOL-032 (Missing has_one)      | 5     | security_config is program singleton, not user PDA |
| SOL-034 (Token-2022 Delegate)  | 18    | Properly validated in exit_check()                 |
| SOL-040 (CpiContext unchecked) | 14    | Program validated via address constraint           |
| SOL-016 (Integer Overflow)     | 42    | Uses checked arithmetic                            |
| SOL-014 (Reinitialization)     | 12    | Uses init constraint properly                      |
| Other                          | 16    | Various context-dependent reasons                  |
| Total                          | 107   |                                                    |

## Appendix F: Agent Analysis Summary

| Agent                     | Raw Findings | True Positives | Critical | High     | Medium   |
|---------------------------|--------------|----------------|----------|----------|----------|
| CPI Security              | 10           | 3              | 1        | 1        | 1        |
| Account Validation        | 10           | 4              | 2        | 1        | 1        |
| Arithmetic                | 10           | 6              | 0        | 1        | 4        |
| Security Layer            | 5            | 5              | 0        | 3        | 1        |
| <b>Deduplicated Total</b> | <b>35</b>    | <b>21</b>      | <b>2</b> | <b>6</b> | <b>7</b> |

## Appendix G: Disclaimer

This security assessment does not guarantee the absence of vulnerabilities. The assessment was limited to the code provided at the time of engagement. New vulnerabilities may be discovered after the assessment date.

The findings and recommendations in this report are provided "as is" without warranty of any kind. The audit scope was defined by the [DeGate LP Handler Audit Requirements](#) document. Business logic in the `instructions/` module was reviewed for account constraints and CPI patterns but NOT for business correctness, as defined in the audit scope.

**Critical Prerequisite:** This security model operates under the assumption that third-party DAPP programs invoked via CPI are **non-malicious**. The security model is NOT designed to defend against malicious third-party DAPPs.

**Report Generated:** February 06, 2026 **Assessment Version:** 3.0 (Comprehensive Audit)